

Persistencia Simple con Spring Data JPA + H2

Automatiza tu capa de datos sin escribir una sola línea de SQL



¿Qué aprenderás hoy?

Objetivos

- Simplificar la persistencia con Spring Data JPA
- Dominar JpaRepository y sus métodos automáticos
- Configurar H2 para desarrollo rápido
- Probar la integración completa con Postman

Agenda

1. El problema del código manual
2. Spring Data JPA: menos código, más productividad
3. Configuración práctica con H2
4. Demo en vivo y buenas prácticas



El Problema Actual

Los repositorios tradicionales requieren implementaciones manuales que consumen tiempo y generan código repetitivo innecesario.

Repositorio en memoria

Datos volátiles que se pierden al reiniciar. Ideal solo para pruebas iniciales.

Implementaciones manuales

EntityManager, consultas JPQL repetitivas, gestión manual de transacciones.

Código duplicado

Los métodos CRUD son prácticamente idénticos en cada repositorio del proyecto.



¿Qué Agrega Spring Data JPA?

Spring Data JPA construye sobre JPA estándar, eliminando código boilerplate y generando implementaciones de repositorio automáticamente.



Interfaces declarativas

Define solo la interfaz; Spring genera la implementación completa en tiempo de ejecución.



Métodos automáticos

Accede a operaciones CRUD sin escribir código: `save()`, `findAll()`, `deleteById()`.



Consultas derivadas

Crea queries complejas solo con el nombre del método: `findByNombreAndEdad()`.

JpaRepository: Tu Mejor Aliado

La interfaz JpaRepository proporciona métodos listos para usar que cubren el 80% de las operaciones de persistencia habituales.

1

Operaciones de lectura

```
findAll()
findById(Long id)
existsById(Long id)
count()
```

2

Operaciones de escritura

```
save(T entity)
saveAll(Iterable<T> entities)
flush()
```

3

Operaciones de eliminación

```
deleteById(Long id)
delete(T entity)
deleteAll()
```

📌 **Ventaja clave:** No necesitas implementar ninguno de estos métodos. Spring Data JPA genera toda la lógica automáticamente al extender la interfaz.

Consultas Derivadas por Nombre

Spring Data JPA analiza el nombre de tu método y genera la consulta SQL correspondiente automáticamente. Es magia basada en convenciones.

Búsqueda por campo

```
findByNombre(String nombre)
```

Genera: `SELECT * FROM usuario WHERE nombre = ?`

Múltiples condiciones

```
findByNombreAndEdad(  
    String nombre,  
    Integer edad)
```

Combina filtros con operadores lógicos AND/OR.

Ordenamiento integrado

```
findByNombreOrderByEdadDesc(  
    String nombre)
```

Añade ordenamiento directamente en el nombre.

Configuración Rápida con H2

H2 es una base de datos en memoria perfecta para desarrollo y pruebas. Se configura en segundos y proporciona una consola web integrada.

Configuración en application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
spring.jpa.show-sql=true
```

01

Agregar dependencia

Incluir H2 en pom.xml con scope runtime.

02

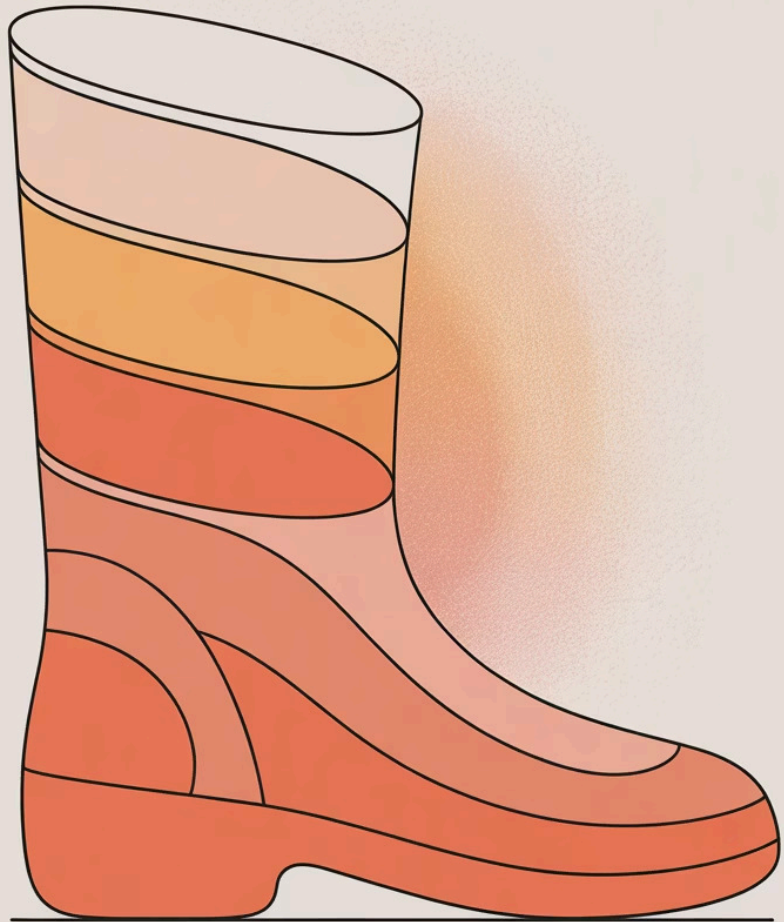
Configurar properties

Definir URL y habilitar la consola web.

03

Acceder a consola

Navegar a localhost:8080/h2-console para ver datos.



Integración con las Capas Existentes

Spring Data JPA se integra perfectamente con tu arquitectura Controller–Service–Repository sin romper la separación de responsabilidades.

1

Controller

Recibe peticiones HTTP y delega al Service. Sin cambios.

2

Service

Contiene lógica de negocio. Llama al Repository. Sin cambios.

3

Repository

Ahora es solo una interfaz que extiende JpaRepository. ¡Simplificado!

4

Base de datos

H2 o cualquier BD relacional. Persistencia real automática.

Demo en Vivo

Transformando el Repositorio

Antes: Implementación manual

```
@Repository
public class UsuarioRepositoryImpl {
    private Map<Long, Usuario> datos;

    public Usuario save(Usuario u) {
        // Código manual...
    }

    public List<Usuario> findAll() {
        // Más código...
    }
}
```

Después: Spring Data JPA

```
@Repository
public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

    // ¡Eso es todo!
    // Métodos CRUD incluidos

}
```

Menos código, más funcionalidad

📌 **Próximos pasos en la demo:** Probar endpoints con Postman (POST para crear, GET para listar), verificar persistencia en consola H2, y observar las consultas SQL generadas automáticamente en logs.

Ventajas y Buenas Prácticas



Desarrollo acelerado

Eliminas código repetitivo y te enfocas en la lógica de negocio única de tu aplicación.



Consistencia garantizada

Todas las operaciones CRUD siguen patrones probados y optimizados por el framework.



Portabilidad entre BDs

Cambia de H2 a PostgreSQL o MySQL solo modificando configuración, sin tocar código.

Recomendaciones importantes

- Usa H2 solo para desarrollo; en producción migra a bases de datos robustas como PostgreSQL
- Aprovecha `@Query` para consultas complejas que no se pueden expresar con nombres de método
- Implementa paginación con `Pageable` para evitar cargar miles de registros en memoria
- Habilita `show-sql` en desarrollo para entender qué queries genera Spring Data JPA

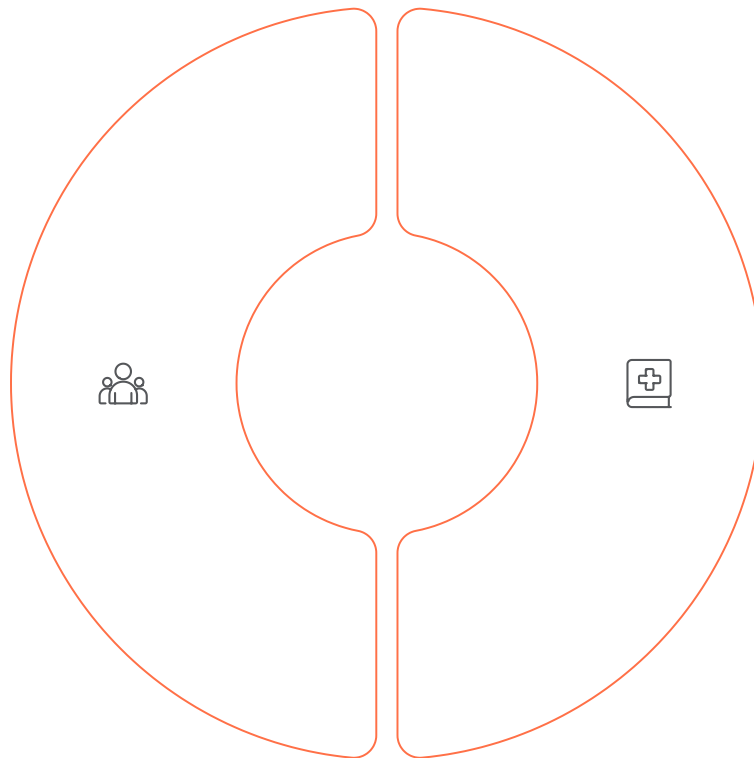
CHECKLIST



Resumen y Próximos Pasos

Aprendiste hoy

- JpaRepository y métodos automáticos
- Consultas derivadas por nombre
- Configuración de H2 en Spring Boot
- Integración completa en tu arquitectura



Practica ahora

- Implementa repositorios para otras entidades
- Crea métodos de búsqueda personalizados
- Explora la consola H2
- Prueba diferentes configuraciones

¡Gracias por seguir la serie de API REST con Spring Boot! Continúa practicando y nos vemos en el próximo video.